

Creación de Formularios

Web Design

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. Introducción a los formularios web
2. Estructura básica de un formulario HTML
3. Elementos de entrada de datos (input)
4. Otros controles de formularios
5. Etiquetas y organización de formularios
6. Atributos de los controles
7. Validación de formularios con HTML5
8. Buenas prácticas y accesibilidad

Introducción a los formularios web

¿Qué es un formulario web?

Un formulario web es un conjunto de elementos HTML que permite recopilar información proporcionada por los usuarios.

Su propósito principal es facilitar la interacción entre el usuario y una aplicación web.

Algunos datos que pueden recopilarse son:

- Información personal.
- Credenciales de acceso.
- Opiniones y comentarios.
- Preferencias y opciones.
- Datos para realizar búsquedas.

¿Dónde encontramos formularios?

Los formularios están presentes en prácticamente cualquier aplicación web.

- **Registro:** creación de cuentas de usuario.
- **Inicio de sesión:** autenticación de usuarios.
- **Contacto:** envío de mensajes y consultas.
- **Encuestas:** recopilación de opiniones.
- **Búsquedas:** localización de información.
- **Compras:** captura de datos de envío y facturación.

Elementos que componen un formulario

Un formulario puede contener diferentes tipos de elementos para recopilar información.

- **Campos de entrada:** permiten escribir información.
- **Casillas de selección:** permiten elegir varias opciones.
- **Botones de opción:** permiten seleccionar una alternativa.
- **Listas desplegables:** presentan opciones predefinidas.
- **Áreas de texto:** permiten introducir textos extensos.
- **Botones:** permiten realizar acciones sobre el formulario.

Cada elemento tiene una función específica y se representa mediante etiquetas HTML.

Ejemplo: formulario de contacto

Un formulario sencillo puede solicitar información básica de un usuario.

```
<form>
    <label>Nombre:</label>
    <input type="text" name="nombre">

    <label>Correo:</label>
    <input type="email" name="correo">

    <label>Mensaje:</label>
    <textarea name="mensaje"></textarea>

    <button type="submit">Enviar</button>
</form>
```

HTML y los formularios

HTML proporciona las etiquetas necesarias para construir formularios y definir sus controles. Algunas de las etiquetas más importantes son:

Etiqueta	Propósito
`<form>`	Define el formulario.
`<input>`	Crea campos de entrada.
`<label>`	Identifica un control.
`<textarea>`	Captura texto multilínea.
`<select>`	Crea una lista de opciones.
`<button>`	Crea un botón.

HTML5 también incorpora atributos que permiten establecer restricciones y validar los datos introducidos.

Objetivo de los formularios HTML5

Al finalizar este tema, se espera que el estudiante sea capaz de:

- Identificar los diferentes controles de un formulario.
- Construir formularios utilizando etiquetas HTML5.
- Seleccionar el tipo de entrada adecuado para cada dato.
- Organizar y agrupar controles.
- Aplicar atributos para establecer restricciones.
- Utilizar los mecanismos de validación proporcionados por HTML5.

Estructura básica de un formulario HTML

La etiqueta `<form>`

La etiqueta `<form>` es el elemento principal para crear formularios en HTML.

Permite agrupar controles que recopilan información del usuario.

Sintaxis básica:

```
<form>
    <!-- Elementos del formulario -->
</form>
```

Los controles incluidos dentro de esta etiqueta forman parte del mismo formulario.

Primer formulario

El siguiente ejemplo muestra un formulario que solicita el nombre y el correo electrónico de un usuario.

```
<form>
    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="correo">Correo:</label>
    <input type="email" id="correo" name="correo">

    <button type="submit">Enviar</button>
</form>
```

- `form`: delimita el formulario.
- `label`: proporciona una descripción del campo.
- `input`: permite capturar información.
- `button`: permite enviar el formulario.

Atributos de la etiqueta `<form>`

La etiqueta `<form>` admite diferentes atributos para definir el comportamiento del formulario.

Los más importantes son:

- **action:** especifica la dirección a la que se enviarán los datos.
- **method:** establece el método HTTP utilizado para enviar la información.
- **autocomplete:** indica si el navegador puede completar automáticamente los campos.
- **enctype:** especifica cómo se codifican los datos al enviarlos.

Atributo action

El atributo `action` especifica la dirección del recurso que recibirá los datos del formulario.

```
<form action="/procesar.php">
    <label for="usuario">Usuario:</label>
    <input type="text" id="usuario" name="usuario">

    <button type="submit">Enviar</button>
</form>
```

En este caso, los datos se enviarán al recurso `/procesar.php`. Si no se especifica `action`, el formulario se envía a la URL del documento actual.

Atributo method

El atributo `method` determina cómo se envían los datos del formulario mediante HTTP.

HTML permite utilizar principalmente dos métodos:

GET
Los datos se incorporan a la URL como parámetros. Son visibles en la barra de direcciones del navegador.

POST
Los datos se envían en el cuerpo de la petición HTTP. No aparecen directamente en la URL.

Si no se especifica el atributo `method`, el navegador utiliza GET.

Método GET

Con GET, los datos del formulario se agregan a la URL como pares de nombre y valor.

```
<form action="/buscar" method="get">
    <label for="producto">Producto:</label>
    <input type="text" id="producto" name="producto">

    <button type="submit">Buscar</button>
</form>
```

Si el usuario escribe “computadora”, la petición puede generar una URL similar a:

`/buscar?producto=computadora`

- Adecuado para búsquedas y consultas.
- Permite guardar y compartir la URL.
- No debe utilizarse para enviar contraseñas.

Método POST

Con POST, los datos se envían en el cuerpo de la petición HTTP, en lugar de agregarse a la URL.

```
<form action="/registro" method="post">
    <label for="usuario">Usuario:</label>
    <input type="text" id="usuario" name="usuario">

    <label for="clave">Contraseña:</label>
    <input type="password" id="clave" name="clave">

    <button type="submit">Registrarse</button>
</form>
```

- Adecuado para registros y envío de información.
- Los datos no aparecen directamente en la URL.
- Para proteger información sensible se debe utilizar HTTPS.

GET vs POST

Característica	GET	POST
Ubicación de datos	URL	Cuerpo HTTP
Visibilidad	En la URL	No visibles en la URL
Marcadores	Permite guardar la consulta	No guarda los datos en la URL
Uso habitual	Búsquedas	Registros
Tamaño	Limitado por la URL	Mayor flexibilidad

Importante: POST no cifra los datos. La protección de la información requiere HTTPS.

Atributo enctype

El atributo `enctype` determina cómo se codifican los datos del formulario cuando se utiliza POST. Los valores más utilizados son:

- `application/x-www-form-urlencoded`: codificación predeterminada.
- `multipart/form-data`: permite enviar archivos.
- `text/plain`: envía los datos como texto plano.

Ejemplo para enviar archivos:

```
<form action="/subir" method="post"
      enctype="multipart/form-data">

    <input type="file" name="archivo">

    <button type="submit">Subir archivo</button>
</form>
```

Atributo autocomplete

Permite indicar si el navegador puede sugerir o completar automáticamente información en los campos del formulario. Admite principalmente los valores:

- `on`: permite el autocompletado.
- `off`: solicita desactivar el autocompletado.

```
<form autocomplete="on">
    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="correo">Correo:</label>
    <input type="email" id="correo" name="correo">
</form>
```

Resumen

La etiqueta `<form>` permite agrupar controles y definir cómo se enviará la información.

Recordemos:

- `action`: destino de los datos.
- `method`: método HTTP utilizado.
- `GET`: envía los datos en la URL.
- `POST`: envía los datos en el cuerpo HTTP.
- `enctype`: formato de codificación.
- `autocomplete`: configuración del autocompletado.

Elementos de entrada de datos (input)

La etiqueta `<input>`

La etiqueta `<input>` permite crear diferentes controles para capturar información del usuario. Su comportamiento depende principalmente del atributo `type`.

```
<input type="tipo" name="nombre">
```

Algunos atributos importantes:

- `type`: determina el tipo de control.
- `name`: identifica el dato al enviarlo.
- `id`: identifica el elemento en el documento.
- `value`: establece un valor inicial.

Entrada de texto: text

El tipo `text` permite introducir texto en una sola línea.

```
<label for="nombre">Nombre:</label>
<input type="text" id="nombre" name="nombre">
```

Se utiliza para capturar:

- Nombres y apellidos.
- Nombres de usuario.
- Direcciones y otros datos de texto.

Entrada de contraseña: password

El tipo `password` permite introducir contraseñas. Los caracteres se muestran ocultos visualmente.

```
<label for="clave">Contraseña:</label>
<input type="password" id="clave" name="clave">
```

- Oculta visualmente los caracteres introducidos.
- No cifra ni protege por sí mismo la contraseña.
- Se utiliza habitualmente en formularios de acceso.

Entrada de correo electrónico: email

El tipo `email` permite capturar direcciones de correo electrónico.

```
<label for="correo">Correo electrónico:</label>
<input type="email" id="correo"
       name="correo" required>
```

- El navegador comprueba que el valor tenga un formato compatible con una dirección de correo.
- Puede presentar sugerencias de autocompletado.
- No comprueba que la dirección realmente exista.

Entrada numérica: number

El tipo `number` permite introducir valores numéricos.

```
<label for="edad">Edad:</label>
<input type="number" id="edad"
       name="edad" min="18"
       max="100" step="1">
```

Sus atributos permiten establecer restricciones:

- `min`: valor mínimo permitido.
- `max`: valor máximo permitido.
- `step`: intervalo entre valores válidos.

Control deslizante: range

El tipo `range` permite seleccionar un valor dentro de un intervalo mediante un control deslizante.

```
<label for="satisfaccion">
    Nivel de satisfacción:
</label>

<input type="range" id="satisfaccion"
       name="satisfaccion" min="0" max="10"
       step="1" value="5">
```

Se utiliza cuando el usuario debe seleccionar un valor aproximado dentro de un rango.

Entrada de fecha: date

El tipo `date` permite seleccionar una fecha mediante un control proporcionado por el navegador.

```
<label for="nacimiento">
    Fecha de nacimiento:
</label>

<input type="date" id="nacimiento"
       name="nacimiento">
```

- Facilita la selección de fechas.
- El formato de presentación depende del navegador y la configuración regional.
- Internamente, el valor se representa como `AAAA-MM-DD`.

Entrada de hora: time

El tipo `time` permite seleccionar una hora.

```
<label for="hora">
    Hora de atención:
</label>

<input type="time" id="hora" name="hora">
```

- Permite seleccionar horas y minutos.
- Su presentación depende del navegador.
- El valor enviado utiliza el formato de 24 horas: `HH:MM`.

Entrada de dirección web: url

El tipo `url` permite introducir direcciones web.

```
<label for="sitio">
    Sitio web:
</label>

<input type="url" id="sitio" name="sitio"
       placeholder="https://ejemplo.com">
```

- Comprueba que el valor tenga un formato compatible con una URL.
- Puede utilizarse para capturar sitios personales, páginas institucionales o enlaces.

Campo de búsqueda: search

El tipo `search` permite introducir términos de búsqueda.

```
<label for="buscar">
    Buscar producto:
</label>

<input type="search" id="buscar" name="buscar"
       placeholder="Escribe un producto">
```

- Está diseñado para búsquedas.
- Su apariencia puede variar entre navegadores.
- Puede incorporar controles para limpiar el contenido.

Campo oculto: hidden

El tipo `hidden` permite incluir información en un formulario sin mostrar un control visible al usuario.

```
<form>
    <input type="hidden"
           name="id_producto"
           value="125">

    <label for="cantidad">Cantidad:</label>
    <input type="number" id="cantidad"
           name="cantidad" value="1">

    <button type="submit">Comprar</button>
</form>
```

Importante: los campos ocultos también pueden ser modificados por el usuario. No deben utilizarse para almacenar información confidencial ni para confiar en datos que deben validarse en el servidor.

Botones de envío y restablecimiento

HTML proporciona tipos de entrada para realizar acciones sobre un formulario.

Envía los datos del formulario: `submit`

```
<input type="submit" value="Enviar">
```

Recupera los valores iniciales de los controles: `reset`

```
<input type="reset" value="Limpiar">
```

Estos controles se utilizan habitualmente al finalizar la captura de información.

Resumen de tipos de input

Tipo	Función
`text`	Texto de una línea.
`password`	Contraseña.
`email`	Correo electrónico.
`number`	Valores numéricos.
`range`	Selección mediante deslizador.
`date`	Fecha.
`time`	Hora.
`url`	Dirección web.
`search`	Términos de búsqueda.
`hidden`	Dato no visible.
`submit`	Envío del formulario.
`reset`	Restablecimiento de controles.

Consideraciones importantes

No todos los datos deben capturarse utilizando el mismo tipo de entrada.

Para seleccionar un control adecuado debemos considerar:

- La naturaleza del dato que se desea capturar.
- El tipo de información que debe introducir el usuario.
- Las restricciones que deben aplicarse.
- Las características de validación que proporciona el navegador.

Elegir correctamente el tipo de `input` facilita la captura de información y mejora la experiencia del usuario.

Otros controles de formularios

Más allá de la etiqueta input

HTML5 proporciona otros elementos que permiten recopilar información de diferentes maneras. Algunos controles no utilizan la etiqueta `<input>`.

Los principales son:

- `<textarea>`: captura texto en varias líneas.
- `<select>`: presenta una lista de opciones.
- `<option>`: define las opciones disponibles.
- `<input type="checkbox">`: permite seleccionar varias alternativas.
- `<input type="radio">`: permite seleccionar una alternativa.
- `<button>`: crea botones para realizar acciones.

Área de texto: textarea

La etiqueta `<textarea>` permite introducir texto en múltiples líneas.

```
<label for="comentario">Comentario:</label>

<textarea id="comentario" name="comentario"
          rows="5" cols="40">
</textarea>
```

Sus atributos principales son:

- `rows`: cantidad visible de filas.
- `cols`: cantidad visible de columnas.
- `name`: nombre del dato enviado.
- `maxlength`: longitud máxima del texto.

Listas desplegables: select y option

La etiqueta `<select>` permite presentar una lista de opciones para que el usuario seleccione una. Las opciones se definen mediante `<option>`.

```
<label for="carrera">Carrera:</label>

<select id="carrera" name="carrera">
    <option value="informatica">Informática</option>
    <option value="biologia">Biología</option>
    <option value="turismo">Turismo</option>
    <option value="acuicultura">Acuicultura</option>
</select>
```

El atributo `value` establece el valor que se enviará cuando el usuario seleccione una opción.

Opción seleccionada por defecto

El atributo `selected` permite establecer una opción seleccionada inicialmente.

```
<label for="semestre">Semestre:</label>

<select id="semestre" name="semestre">
    <option value="">Selecciona un semestre</option>
    <option value="1">Primero</option>
    <option value="2">Segundo</option>
    <option value="3" selected>Tercero</option>
    <option value="4">Cuarto</option>
</select>
```

En este ejemplo, la opción Tercero aparece seleccionada inicialmente.

Selección múltiple: select

El atributo `multiple` permite seleccionar más de una opción dentro de una lista.

```
<label for="materias">Materias de interés:</label>

<select id="materias"
        name="materias"
        multiple>
    <option value="html">HTML</option>
    <option value="php">PHP</option>
    <option value="python">Python</option>
    <option value="sql">SQL</option>
</select>
```

- Permite seleccionar varias opciones.
- La forma de realizar la selección depende del navegador y del sistema operativo.
- Para enviar varias opciones, el servidor debe procesar los valores correspondientes.

Casillas de selección: checkbox

El tipo **checkbox** permite seleccionar una o varias opciones de manera independiente.

```
<p>Lenguajes de programación:</p>

<input type="checkbox" id="c" name="lenguaje"
       value="C">
<label for="c">C</label>

<input type="checkbox" id="python" name="lenguaje"
       value="Python">
<label for="python">Python</label>

<input type="checkbox" id="java" name="lenguaje"
       value="Java">
<label for="java">Java</label>
```

Cada casilla puede marcarse o desmarcarse independientemente de las demás.

Características de checkbox

Las casillas de selección se utilizan cuando el usuario puede elegir varias alternativas. Algunas aplicaciones son:

- Selección de intereses.
- Elección de servicios.
- Preferencias de usuario.
- Aceptación de condiciones.

Consideraciones:

- Una casilla marcada envía su valor.
- Una casilla desmarcada no envía ningún valor.
- El atributo `checked` permite marcar una casilla inicialmente.

Botones de opción: radio

El tipo `radio` permite seleccionar una sola alternativa dentro de un grupo. Para formar un grupo, todos los controles deben compartir el mismo atributo `name`.

```
<p>Turno:</p>

<input type="radio" id="matutino"
       name="turno" value="matutino">
<label for="matutino">Matutino</label>

<input type="radio" id="vespertino"
       name="turno" value="vespertino">
<label for="vespertino">Vespertino</label>

<input type="radio" id="nocturno"
       name="turno" value="nocturno">
<label for="nocturno">Nocturno</label>
```

Características de radio

Los botones de opción se utilizan cuando el usuario debe elegir una sola alternativa. Ejemplos de aplicación:

- Seleccionar un turno.
- Elegir un método de pago.
- Indicar el nivel de estudios.
- Seleccionar una modalidad.

Consideraciones:

- Los controles del mismo grupo comparten el atributo `name`.
- Solamente una opción puede permanecer seleccionada.
- El atributo `checked` establece la selección inicial.

Checkbox vs Radio

Ambos controles permiten seleccionar opciones, pero tienen comportamientos diferentes.

Característica	Checkbox	Radio
Selección	Varias opciones	Una opción por grupo
Estado	Independiente	Excluyente
Uso	Preferencias	Alternativas únicas
Agrupación	No requiere un mismo nombre	Requiere el mismo `name`

La elección depende de si se permite seleccionar una o varias alternativas.

Botones: button

La etiqueta `<button>` permite crear botones dentro de un formulario. Existen tres tipos principales:

- `submit`: envía el formulario.
- `reset`: restablece los controles.
- `button`: botón sin acción predeterminada.

```
<form>
    <input type="text" name="nombre">

    <button type="submit">Enviar</button>
    <button type="reset">Limpiar</button>
    <button type="button">Botón normal</button>
</form>
```

Es recomendable especificar siempre el atributo `type`.

Resumen de controles

Elemento	Función
`textarea`	Capturar texto multilínea.
`select`	Presentar una lista de opciones.
`option`	Definir una opción.
`checkbox`	Seleccionar varias alternativas.
`radio`	Seleccionar una alternativa.
`button`	Crear botones.

Cada control responde a una necesidad específica de captura de información.

Etiquetas y organización de formularios

Organización de un formulario

Un formulario debe organizar sus controles de manera clara para facilitar la captura de información.

HTML proporciona elementos que permiten identificar, describir y agrupar los controles.

Los principales son:

- `<label>`: identifica un control.
- `<fieldset>`: agrupa controles relacionados.
- `<legend>`: proporciona un título al grupo.
- `placeholder`: muestra una indicación dentro del campo.
- `value`: establece el valor inicial de un control.

Etiqueta label

La etiqueta `<label>` permite asociar un texto descriptivo con un control del formulario.

```
<label for="nombre">Nombre completo:</label>
<input type="text" id="nombre" name="nombre">
```

- El atributo `for` debe coincidir con el `id` del control.
- Al hacer clic en la etiqueta, el navegador enfoca el control asociado.
- Facilita la interacción con los formularios.
- Mejora la accesibilidad, especialmente para lectores de pantalla.

Asociación de label con un control

Existen dos formas habituales de asociar una etiqueta con un control.

1. Asociación mediante for e id

```
<label for="correo">Correo:</label>
<input type="email" id="correo" name="correo">
```

2. Asociación mediante anidamiento

```
<label>
    Correo:
    <input type="email" name="correo">
</label>
```

Ambas formas permiten asociar el texto descriptivo con el control correspondiente.

¿Por qué utilizar label?

Utilizar etiquetas descriptivas proporciona varias ventajas:

- Identifica claramente qué información se solicita.
- Permite seleccionar el control haciendo clic en su etiqueta.
- Facilita la navegación y comprensión del formulario.
- Mejora la accesibilidad para personas que utilizan tecnologías de asistencia.

Buena práctica
Todo control de captura de información debe contar con una etiqueta descriptiva asociada, siempre que corresponda.

Agrupación de controles: fieldset

La etiqueta `<fieldset>` permite agrupar controles que están relacionados entre sí.

```
<form>
    <fieldset>
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre">

        <label for="correo">Correo:</label>
        <input type="email" id="correo" name="correo">
    </fieldset>
</form>
```

El navegador normalmente representa el grupo mediante un borde alrededor de los controles.

Título de un grupo: legend

La etiqueta `<legend>` permite establecer un título o descripción para un grupo definido mediante `<fieldset>`.

```
<fieldset>
    <legend>Datos personales</legend>

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="edad">Edad:</label>
    <input type="number" id="edad" name="edad">
</fieldset>
```

El título permite identificar fácilmente la información que se solicita dentro del grupo.

Agrupación de diferentes secciones

Un formulario extenso puede dividirse en grupos temáticos para mejorar su organización.

```
<form>
    <fieldset>
        <legend>Datos personales</legend>
        <input type="text" name="nombre"
               placeholder="Nombre completo">
        <input type="email" name="correo"
               placeholder="Correo">
    </fieldset>

    <fieldset>
        <legend>Datos académicos</legend>
        <input type="text" name="carrera"
               placeholder="Carrera">
        <input type="number" name="semestre"
               placeholder="Semestre">
    </fieldset>

    <button type="submit">Enviar</button>
</form>
```

Atributo placeholder

El atributo `placeholder` muestra un texto de ayuda dentro de un campo de entrada.

```
<label for="usuario">Nombre de usuario:</label>

<input type="text" id="usuario"
       name="usuario" placeholder="Ej. juan123">
```

- Proporciona una indicación sobre el dato esperado.
- Desaparece cuando el usuario introduce información.
- No sustituye a la etiqueta `label`.
- No debe utilizarse como única descripción del campo.

Atributo value

El atributo `value` permite establecer un valor inicial para determinados controles.
Ejemplo con input:

```
<label for="pais">País:</label>

<input type="text" id="pais" name="pais" value="México">
```

Ejemplo con option:

```
<select name="pais">
    <option value="mx">México</option>
    <option value="es">España</option>
    <option value="ar">Argentina</option>
</select>
```

El valor es el dato que se envía al servidor cuando el control participa en el envío del formulario.

Diferencia entre placeholder y value

Aunque ambos atributos pueden mostrar texto dentro de un campo, su comportamiento es diferente.

Característica	placeholder	value
Propósito	Mostrar una indicación	Establecer un valor
Contenido	Texto de ayuda	Dato inicial
Al escribir	Desaparece	Se reemplaza o modifica
Envío	No se envía como valor	Se envía si el control es válido y tiene nombre

```
<input type="text" placeholder="Escribe tu nombre">
<input type="text" value="Juan">
```

Ejemplo integrador: formulario organizado

```
<form>
    <fieldset>
        <legend>Datos personales</legend>

        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre"
               name="nombre" placeholder="Nombre completo">

        <label for="correo">Correo:</label>
        <input type="email" id="correo"
               name="correo">
    </fieldset>

    <fieldset>
        <legend>Datos académicos</legend>

        <label for="carrera">Carrera:</label>
        <select id="carrera" name="carrera">
            <option value="tic">TIC</option>
            <option value="bio">Biología</option>
        </select>
    </fieldset>

    <button type="submit">Registrar</button>
</form>
```

Resumen

Los elementos de organización permiten construir formularios más claros y accesibles.

- `label`: describe y asocia un control.
- `fieldset`: agrupa controles relacionados.
- `legend`: identifica el grupo.
- `placeholder`: proporciona una indicación.
- `value`: establece un valor inicial.

Una buena organización facilita tanto la captura de información como la comprensión del formulario.

Atributos de los controles

Atributos de los controles HTML

Los controles de un formulario pueden configurarse mediante atributos que determinan su comportamiento y establecen restricciones sobre los datos introducidos.

Algunos atributos permiten:

- Identificar los controles.
- Establecer valores iniciales.
- Definir campos obligatorios.
- Restringir la modificación de información.
- Limitar los valores y la longitud de los datos.
- Controlar el autocompletado.

Estos atributos permiten mejorar la captura de información sin necesidad de utilizar JavaScript.

Atributos name e id

Aunque ambos atributos identifican elementos, tienen propósitos diferentes. **name**: Identifica el dato cuando se envía el formulario al servidor. **id**: Identifica de manera única un elemento dentro del documento HTML.

```
<label for="nombre">Nombre:</label>

<input type="text" id="nombre" name="nombre">
```

- `id` permite asociar el control con su etiqueta.
- `name` determina el nombre del parámetro enviado.

Diferencia entre name e id

Consideremos el siguiente formulario:

```
<input type="text" id="usuario" name="login">
```

Si el usuario introduce el valor `carlos`, el navegador enviará un parámetro similar a:

```
login=carlos
```

- El atributo `id` debe ser único en el documento.
- El atributo `name` puede repetirse, por ejemplo, en grupos de botones de opción.
- Un control sin `name` normalmente no participa en el envío de datos.

Atributo required

El atributo `required` establece que un campo debe contener información antes de enviar el formulario.

```
<form>
    <label for="nombre">Nombre:</label>
    <input type="text"
           id="nombre"
           name="nombre"
           required>

    <label for="correo">Correo:</label>
    <input type="email"
           id="correo"
           name="correo"
           required>

    <button type="submit">Enviar</button>
</form>
```

Si el usuario intenta enviar el formulario sin completar los campos obligatorios, el navegador impedirá el envío.

Atributos readonly y disabled

Ambos atributos impiden que el usuario modifique directamente el contenido de un control, pero su comportamiento es diferente.

readonly
El campo no puede modificarse, pero su valor puede seleccionarse y normalmente se incluye en el envío.

disabled
El control queda deshabilitado y no puede interactuarse con él. Su valor no se incluye en el envío del formulario.

Se utilizan cuando necesitamos mostrar información que el usuario no debe modificar o cuando un control no está disponible.

Ejemplo: readonly y disabled

```
<form>
    <label for="matricula">Matrícula:</label>
    <input type="text"
           id="matricula"
           name="matricula"
           value="20260015"
           readonly>

    <label for="grupo">Grupo:</label>
    <input type="text"
           id="grupo"
           name="grupo"
           value="A"
           disabled>

    <button type="submit">Enviar</button>
</form>
```

En este ejemplo:

- La matrícula se envía, aunque no puede modificarse.
- El grupo está deshabilitado y no se envía.

Atributos min, max y step

Estos atributos permiten establecer restricciones para controles numéricos y algunos controles de fecha y hora.

- **min:** establece el valor mínimo permitido.
- **max:** establece el valor máximo permitido.
- **step:** define los incrementos permitidos.

```
<input type="number"
       min="0"
       max="100"
       step="5">
```

Los valores válidos serán 0, 5, 10, 15, ..., 100.

Ejemplo: restricciones numéricas

```
<label for="calificacion">
    Calificación:
</label>

<input type="number" id="calificacion" name="calificacion"
       min="0" max="100" step="1" required>
```

En este caso:

- La calificación mínima es 0.
- La calificación máxima es 100.
- Los incrementos son de una unidad.
- El campo es obligatorio.

El navegador puede impedir el envío si el valor no cumple estas restricciones.

Atributos minlength y maxlength

Permiten establecer restricciones sobre la longitud del texto introducido.

- **minlength:** define la cantidad mínima de caracteres.
- **maxlength:** define la cantidad máxima de caracteres.

```
<input type="text" minlength="3" maxlength="20">
```

El usuario deberá introducir al menos 3 caracteres y no podrá superar los 20 caracteres.

Ejemplo: longitud de una contraseña

```
<label for="clave">Contraseña:</label>

<input type="password" id="clave" name="clave"
       minlength="8" maxlength="20" required>
```

El navegador verificará que:

- El campo no esté vacío.
- La contraseña tenga al menos 8 caracteres.
- La contraseña no supere los 20 caracteres.

Importante: estas restricciones no sustituyen las políticas de seguridad ni la validación del servidor.

Atributo pattern

El atributo `pattern` permite establecer una expresión regular que debe cumplir el valor introducido.

Se utiliza principalmente en controles de tipo:

- `text`
- `search`
- `tel`
- `url`
- `password`

Atributo pattern

Permite definir formatos específicos para los datos, por ejemplo:

- Códigos postales.
- Matrículas.
- Números telefónicos con un formato determinado.
- Identificadores institucionales.

Ejemplo: validación mediante pattern

Supongamos que queremos capturar una matrícula de ocho dígitos numéricos.

```
<label for="matricula">Matrícula:</label>

<input type="text" id="matricula" name="matricula" pattern="[0-9]{8}"
       title="Introduce exactamente 8 dígitos" required>
```

La expresión:

```
[0-9]{8}
```

significa que deben introducirse exactamente ocho dígitos del 0 al 9. El atributo `title` proporciona una indicación sobre el formato esperado.

Atributo autocomplete

El atributo `autocomplete` permite indicar si el navegador puede sugerir información previamente introducida.

Sus valores principales son:

- `on`: permite el autocompletado.
- `off`: solicita desactivarlo.

También permite especificar el tipo de información esperada mediante valores como:

- `name`: nombre completo.
- `email`: correo electrónico.
- `tel`: teléfono.
- `street-address`: dirección.
- `postal-code`: código postal.

Ejemplo: autocomplete

```
<form autocomplete="on">

    <label for="nombre">Nombre:</label>
    <input type="text"
           id="nombre"
           name="nombre"
           autocomplete="name">

    <label for="correo">Correo:</label>
    <input type="email"
           id="correo"
           name="correo"
           autocomplete="email">

    <button type="submit">Enviar</button>

</form>
```

El navegador puede utilizar información previamente guardada para sugerir valores en los campos.

Resumen de atributos

Atributo	Función
`id`	Identifica un elemento.
`name`	Identifica el dato enviado.
`required`	Establece un campo obligatorio.
`readonly`	Impide modificar el valor.
`disabled`	Deshabilita el control.
`min`	Define el valor mínimo.
`max`	Define el valor máximo.
`step`	Define los incrementos permitidos.
`minlength`	Longitud mínima de texto.
`maxlength`	Longitud máxima de texto.
`pattern`	Establece un formato específico.
`autocomplete`	Configura el autocompletado.

Consideraciones importantes

Los atributos permiten establecer restricciones directamente en HTML, pero no garantizan por sí solos la integridad de la información.

- El usuario puede modificar los datos antes de enviarlos.
- La validación del navegador puede ser omitida.
- El servidor debe verificar nuevamente los datos recibidos.
- Los campos ocultos y deshabilitados no deben considerarse mecanismos de seguridad.

Principio fundamental
La validación del lado del cliente facilita la captura de información; la validación del lado del servidor es indispensable para proteger los datos.

Validación de formularios con HTML5

¿Qué es la validación de formularios?

La validación consiste en comprobar que los datos introducidos por el usuario cumplan determinadas condiciones antes de aceptar el envío del formulario.

HTML5 incorpora mecanismos de validación que permiten establecer restricciones sin utilizar JavaScript.

Principales objetivos son:

- Evitar campos obligatorios vacíos.
- Comprobar el formato de los datos.
- Establecer límites numéricos.
- Restringir la longitud del texto.
- Solicitar información con formatos específicos.

¿Cómo funciona la validación HTML5?

Cuando el usuario intenta enviar un formulario, el navegador comprueba los controles que participan en la validación.

El proceso general es:

1. El usuario completa los campos del formulario.
2. Presiona el botón de envío.
3. El navegador verifica las restricciones definidas.
4. Si existe algún error, muestra un mensaje y detiene el envío.
5. Si los datos cumplen las restricciones, permite continuar con el envío.

La validación se realiza automáticamente mediante el navegador.

Validación de campos obligatorios

El atributo `required` permite establecer que un campo debe contener información.

```
<form>
    <label for="nombre">Nombre:</label>
    <input type="text"
           id="nombre"
           name="nombre"
           required>

    <label for="correo">Correo:</label>
    <input type="email"
           id="correo"
           name="correo"
           required>

    <button type="submit">Enviar</button>
</form>
```

Si el usuario intenta enviar el formulario sin completar alguno de estos campos, el navegador impedirá el envío.

Validación del tipo de dato

Algunos tipos de `input` incorporan validaciones específicas según la información que se desea capturar.

```
<form>
    <label for="correo">Correo electrónico:</label>
    <input type="email" id="correo"
           name="correo" required>

    <label for="sitio">Página web:</label>
    <input type="url" id="sitio"
           name="sitio">

    <label for="edad">Edad:</label>
    <input type="number" id="edad"
           name="edad">

    <button type="submit">Enviar</button>
</form>
```

El navegador comprueba que los valores tengan un formato compatible con el tipo de control.

Validación de correo electrónico

El control `type="email"` comprueba que el dato introducido tenga una estructura compatible con una dirección de correo electrónico.

Algunos ejemplos:

Entrada	Resultado esperado
usuario@dominio.com	Formato válido
nombre.apellido@umar.mx	Formato válido
usuario@	Formato inválido
usuario.dominio.com	Formato inválido
@dominio.com	Formato inválido

Importante: HTML5 verifica el formato, pero no comprueba que la dirección exista o que el usuario tenga acceso a ella.

Validación de valores numéricos

Los atributos `min`, `max` y `step` permiten establecer restricciones sobre valores numéricos.

```
<label for="calificacion">
    Calificación:
</label>

<input type="number"
       id="calificacion"
       name="calificacion"
       min="0"
       max="100"
       step="1"
       required>
```

En este ejemplo:

- Se permiten valores entre 0 y 100.
- Los incrementos son de una unidad.
- El campo es obligatorio.

Validación de longitud

Los atributos `minlength` y `maxlength` permiten establecer límites en la cantidad de caracteres.

```
<label for="usuario">Usuario:</label>

<input type="text"
       id="usuario"
       name="usuario"
       minlength="5"
       maxlength="15"
       required>
```

El navegador comprueba que el usuario introduzca:

- Al menos 5 caracteres.
- Como máximo 15 caracteres.
- Algún contenido, debido a `required`.

Validación mediante pattern

El atributo `pattern` permite definir una expresión regular que establece el formato que debe cumplir un dato.

Algunos ejemplos de aplicación:

- Matrículas universitarias.
- Códigos postales.
- Identificadores de estudiantes.
- Números telefónicos con un formato específico.

La expresión regular se evalúa cuando el usuario intenta enviar el formulario.

Este atributo se aplica a controles como `text`, `tel`, `search`, `url` y `password`.

Ejemplo de validación con pattern

Supongamos que una institución utiliza matrículas de exactamente ocho dígitos.

```
<label for="matricula">Matrícula:</label>

<input type="text" id="matricula" name="matricula" pattern="[0-9]{8}"
       title="Introduce exactamente 8 dígitos" required>
```

Interpretación de la expresión:

- `[0-9]`: acepta cualquier dígito del 0 al 9.
- `{8}`: exige exactamente ocho repeticiones.

Ejemplo válido: `20260015`.

Ejemplo inválido: `202615`.

Mensajes de validación

Cuando un control no cumple las restricciones, el navegador puede mostrar mensajes de error automáticamente.

Algunos ejemplos:

- Campo obligatorio vacío.
- Dirección de correo electrónico inválida.
- Número fuera del intervalo permitido.
- Texto demasiado corto.
- Formato que no coincide con el patrón establecido.

Los mensajes dependen del navegador y de su configuración regional. El atributo `title` puede proporcionar información adicional cuando se utiliza `pattern`.

Desactivar la validación automática

HTML5 permite desactivar la validación automática de un formulario mediante el atributo `novalidate`.

```
<form novalidate>
    <label for="correo">Correo:</label>

    <input type="email"
           id="correo"
           name="correo"
           required>

    <button type="submit">Enviar</button>
</form>
```

Aunque el formulario contiene restricciones, el navegador no ejecutará la validación interactiva automática al enviarlo.

Este atributo no elimina los datos ni las restricciones declaradas en los controles.

Ejemplo integrador: formulario validado

```
<form action="/registro" method="post">

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre"
           name="nombre" required
           minlength="3" maxlength="40">

    <label for="correo">Correo:</label>
    <input type="email" id="correo"
           name="correo" required>

    <label for="edad">Edad:</label>
    <input type="number" id="edad"
           name="edad" min="18" max="100"
           required>

    <label for="matricula">Matrícula:</label>
    <input type="text" id="matricula"
           name="matricula"
           pattern="[0-9]{8}" required>

    <button type="submit">Registrar</button>

</form>
```

¿Qué valida nuestro formulario?

En el ejemplo anterior se establecieron las siguientes restricciones:

Campo	Restricción
Nombre	Obligatorio, entre 3 y 40 caracteres.
Correo	Obligatorio y formato de email.
Edad	Obligatoria, entre 18 y 100.
Matrícula	Obligatoria, exactamente 8 dígitos.

Todas estas restricciones se establecen directamente mediante atributos HTML5, sin utilizar JavaScript.

Limitaciones de la validación HTML5

La validación realizada por el navegador es útil, pero no debe considerarse suficiente para garantizar la integridad de los datos.

Algunas consideraciones:

- El usuario puede modificar el código HTML.
- La validación del navegador puede desactivarse.
- Un cliente puede enviar peticiones sin utilizar el formulario.
- Las restricciones HTML no garantizan que los datos sean correctos o verdaderos.

Resumen

HTML5 proporciona mecanismos de validación que facilitan la captura de información.

- `required`: campos obligatorios.
- `type`: validación según el tipo de dato.
- `min` y `max`: límites numéricos.
- `minlength` y `maxlength`: longitud del texto.
- `pattern`: formato mediante expresiones regulares.
- `novalidate`: desactiva la validación automática.

La validación HTML5 mejora la experiencia del usuario, pero debe complementarse con validación del lado del servidor.

Buenas prácticas y accesibilidad

Buenas prácticas al crear formularios

Al diseñar un formulario debemos procurar que sea sencillo de entender y utilizar.

- Solicitar únicamente la información necesaria.
- Utilizar etiquetas descriptivas para cada campo.
- Organizar los controles en un orden lógico.
- Agrupar los campos relacionados.
- Indicar claramente cuáles son obligatorios.
- Proporcionar instrucciones cuando sean necesarias.
- Utilizar controles HTML adecuados para cada tipo de dato.

Etiquetas claras y descriptivas

Cada control debe tener una etiqueta que indique qué información se espera del usuario.

Ejemplo recomendado

```
<label for="correo">Correo electronico:</label>
<input type="email" id="correo" name="correo">
```

Evitar

Utilizar únicamente el atributo `placeholder` como sustituto de una etiqueta.

El placeholder puede desaparecer cuando el usuario comienza a escribir y no siempre es suficiente para identificar el campo.

Organización de los campos

Los controles relacionados pueden agruparse mediante las etiquetas `fieldset` y `legend`.

```
<fieldset>
    <legend>Datos personales</legend>

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="edad">Edad:</label>
    <input type="number" id="edad" name="edad">
</fieldset>
```

Esta organización facilita la comprensión del formulario, especialmente cuando contiene muchos campos.

Indicar los campos obligatorios

Es importante informar al usuario qué campos debe completar necesariamente.

Ejemplo

```
<label for="nombre">Nombre (obligatorio):</label>
<input type="text" id="nombre"
       name="nombre" required>
```

- Utilizar el atributo `required` para establecer la obligatoriedad.
- Informar visualmente cuáles campos son obligatorios.
- No marcar todos los campos como obligatorios si no es necesario.

Accesibilidad en formularios

La accesibilidad permite que personas con diferentes capacidades puedan utilizar los formularios.

Algunas recomendaciones son:

- Asociar cada campo con su etiqueta mediante `for` e `id`.
- Utilizar elementos HTML semánticos.
- Mantener un orden lógico de lectura y navegación.
- Permitir la navegación mediante el teclado.
- Agrupar controles relacionados con `fieldset` y `legend`.
- Utilizar instrucciones y mensajes comprensibles.
- Evitar depender exclusivamente del color para transmitir información.

Navegación mediante teclado

Un formulario debe poder utilizarse sin necesidad de un ratón.

- La tecla `Tab` permite avanzar entre los controles.
- La combinación `Shift + Tab` permite regresar.
- Las teclas de dirección permiten seleccionar opciones en determinados controles.
- La tecla `Enter` puede enviar un formulario.
- La tecla `Espacio` permite activar casillas de verificación.

Recomendación
Mantener el orden natural de navegación y no modificarlo innecesariamente.

Mensajes e instrucciones comprensibles

Los mensajes deben orientar al usuario sobre cómo completar correctamente el formulario.

Mensaje poco claro	Mensaje descriptivo
Error en el campo.	Ingresa una dirección de correo electrónico válida.

También es recomendable:

- Explicar el formato esperado de los datos.
- Evitar instrucciones ambiguas.
- Utilizar mensajes breves y específicos.

Otras recomendaciones

- Utilizar nombres descriptivos en los atributos `name`.
- Aprovechar el atributo `autocomplete` cuando corresponda.
- Definir correctamente el idioma del documento HTML.
- Evitar formularios excesivamente largos.
- Utilizar botones con textos claros, como “Registrar” o “Guardar”.
- Comprobar que los controles sean fáciles de identificar y utilizar.
- Probar el formulario con diferentes navegadores y dispositivos.

Resumen: buenas prácticas y accesibilidad

- Un formulario debe ser claro, sencillo y organizado.
- Cada control debe tener una etiqueta asociada.
- Los campos relacionados deben agruparse.
- La obligatoriedad de los campos debe comunicarse.
- La navegación debe funcionar mediante teclado.
- Los mensajes deben orientar al usuario.
- La accesibilidad debe considerarse desde el diseño.

Idea principal
Un buen formulario no solamente recopila información: facilita que todas las personas puedan proporcionarla correctamente.